

Draw.io Model

A Nasdanika Zephyr Book

Pavel Vlasov

Version 2026.11.0, 2026-11-07

Table of Contents

Colophon . . . 1

Preface . . . 2

Who this book is for . . . 2

A book that stands on its own . . . 2

What you need . . . 3

Conventions . . . 3

I: The Why . . . 5

1. Draw.io diagrams as data . . . 6

1.1. The style string in miniature . . . 6

1.2. The API is the loader, the model is the resource . . . 6

1.3. What stays Draw.io . . . 7

1.4. Fidelity to the format, not to the pixels . . . 7

2. One family of diagram models . . . 8

2.1. A page is a slide is a diagram . . . 8

2.2. Why Draw.io is the default surface . . . 8

2.3. Give and take . . . 9

3. What else there is . . . 10

3.1. Draw.io itself . . . 10

3.2. Parsers and one-off converters . . . 10

3.3. Diagrams as code . . . 10

3.4. Modeling tools . . . 10

3.5. The real competitor . . . 11

4. What it is for . . . 12

4.1. Queries and checks . . . 12

4.2. Generated diagrams . . . 12

4.3. Restyling . . . 12

4.4. Documentation sites . . . 12

4.5. Diagrams mapped to models . . . 13

4.6. Animated diagrams . . . 13

4.7. Regeneration without losing human work . . . 13

II: The How . . . 14

5. Getting started . . . 15

5.1. Adding the model . . . 15

5.2. Files a resource set loads . . . 16

5.3. How the right handler is found . . . 16
5.4. What the build does . . . 16
6. The model . . . 17
6.1. The classes . . . 17
6.2. What maps where . . . 18
6.3. Label and name . . . 18
6.4. Style . . . 18
6.5. Links . . . 19
7. Loading and reading . . . 20
7.1. Loading . . . 20
7.2. Reading elements . . . 20
7.3. Tags and links . . . 21
7.4. What this edition does not load . . . 22
8. Changing and saving . . . 23
8.1. How a document is written . . . 23
8.2. Images . . . 24
9. Generating, laying out and rendering . . . 25
9.1. Generating . . . 25
9.2. Laying out . . . 26
9.3. Rendering . . . 27
III: Looking Ahead . . . 29
10. Looking ahead . . . 30
10.1. Custom actions and animations as objects . . . 30
10.2. Typed styles . . . 30
10.3. Connection points as ports . . . 30
10.4. Links across documents . . . 30
10.5. Merging generated and arranged diagrams . . . 31
10.6. Mapping to models . . . 31
Appendix A: Reference . . . 32
A.1. Model classes . . . 32
A.2. Enumerations . . . 33
A.3. Java classes . . . 33
Appendix B: Building this book, and its figure . . . 34
B.1. The tests that render . . . 34
B.2. The figure . . . 34
Appendix C: Glossary . . . 35

Colophon

Draw.io Model: A Nasdanika Zephyr Book

Edition 2026.11.0, built on 2026-11-07.

This edition describes the Draw.io model of the Nasdanika SDK Runtime 2026.11.0, Maven coordinates `org.nasdanika.sdk.runtime.models.drawio:model:2026.11.0`, namespace URI `runtime.sdk.nasdanika.org/models/drawio`. Every listing in it is included from the module's tests or test resources, which are compiled and run when the book is built. Its figure was generated by one of those tests, laid out with ELK and rendered by Draw.io's own viewer, and it is a PNG image with the Draw.io document embedded.

Copyright © 2026 Pavel Vlasov.

The text and figures of this book are licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). You may share and adapt them for any purpose, commercial purposes included, provided you give appropriate credit, provide a link to the license, and indicate if you made changes. A suggested credit: "Pavel Vlasov, *Draw.io Model: A Nasdanika Zephyr Book*, licensed under CC BY 4.0", with a link to github.com/Nasdanika-SDK/runtime.

The code is licensed under the Eclipse Public License 2.0, the license of the runtime. In addition, you may use the code of this book's listings in your own programs and documentation without asking for permission and without attribution.

Neither license covers the Nasdanika name or logo: CC BY 4.0 grants no trademark rights.

draw.io and diagrams.net are trademarks of JGraph Ltd. Eclipse, EMF and ELK are trademarks of the Eclipse Foundation. Visio and PowerPoint are trademarks of Microsoft Corporation. Java is a trademark of Oracle and its affiliates. Other names may be trademarks of their owners. Their use here identifies the technologies the model relates to and implies no endorsement.

The source of this book, its tests and the model it describes are at github.com/Nasdanika-SDK/runtime, in `models/drawio/model`.

Typeset with Asciidoctor PDF in Noto Sans and M+ 1mn, in the look of the Nasdanika Zephyr Books.

Preface

This book is about Draw.io diagrams as an EMF model. A `.drawio` file, or a PNG or SVG image that Draw.io exported with the diagram in it, loads into a resource set as a document of pages, layers, shapes and connections, each with its style, properties, tags and link. Programs read it, change it, generate it and save it, and people keep drawing in Draw.io.

The first part of the book is the why: why a diagram should be a model, what the model adds to a file, where it sits among the runtime's diagram models, and what it is good for. The second part is the how, for Java developers, and ends with a diagram generated from the running JVM, laid out with ELK and drawn by Draw.io's viewer. The third says what the model is designed to do and does not do yet.

Who this book is for

Architects and analysts who keep diagrams in Draw.io and want programs to read, check and generate them. Part one is for you, and needs no code.

Developers who read Draw.io documents, generate them, or connect them to the models of the systems they describe. Read the book in order.

A book that stands on its own

This book is one of the Nasdanika Zephyr Books: in-depth manuals, one per module of the Nasdanika SDK Runtime, in the manner of IBM Redbooks. Two of them are close to this one. *Draw.io* is the book of the runtime's Java API for Draw.io documents, which this model loads and saves with, and of rendering diagrams with Draw.io's viewer. *Diagram Model* is the book of the notation-neutral model this one extends. You do not need either to read this one: where the books touch, this one repeats what you need.

The series is named after the Bootswatch theme it is styled with, Zephyr, and has nothing to do with the Zephyr real-time operating system or with SmartBear's Zephyr test management.

The model comes from the Draw.io model of Nasdanika's model collection, rebuilt on the runtime's presentation and diagram models. This book replaces the model's readme, and keeps its argument.

What you need

- **A JDK for Java 25** and **Maven 3.9.16 or later**.
- The model's Java module is `org.nasdanika.sdk.runtime.models.drawio`, its Maven coordinates are `org.nasdanika.sdk.runtime.models.drawio:model:2026.11.0`, and its namespace URI is `runtime.sdk.nasdanika.org/models/drawio`.
- **Draw.io**, to draw: the desktop application, the web application at www.drawio.com, or the Draw.io extension for Visual Studio Code.
- Some familiarity with EMF helps in part two: an EMF model is a set of Java interfaces generated from a metamodel, with containment, references, reflection and persistence for free.

Conventions

- **Every listing is included from a test** of the module, or from a resource that a test loads, and is compiled and run when the book is built. Callouts, such as <1>, explain the lines they mark.
- **The figure is a diagram.** It is a PNG image with the Draw.io document embedded: open it in Draw.io to edit it, or load it into a resource set.
- **Status labels** mark what this edition does not have yet: **PLANNED** for designed but not built, **SKETCH** for partly built.
- **Voice.** I write in the first person.

> Editor's notes record inconsistencies between the model and its documentation found while writing. They are shown in builds for the author and left out of published editions.



The readme this book replaces described the model in Nasdanika's model collection: a model tower with floors (C4, architecture, threat, work, governance, nxcore), `DrawioDocument`, `DrawioNode` and `DrawioConnection` class names, a derived read-only `Style` object with typed `NodeStyle` and `ConnectionStyle` subclasses and enumerations, `ConnectionPoint` as a port, loading of custom actions, and the sibling models for Visio, PowerPoint, PlantUML, ODP, PDF and Sprotty. The runtime's model has a style map and no typed styles, keeps connection points in the style, loads custom actions as text, and its siblings are the presentation and diagram models. Visio, PowerPoint and Excel models were postponed on 2026-

> 10-25. This book describes the model as it is, and the rest as planned.

Part I: The Why

Why a Draw.io diagram should be a model and not only a file, what a model adds to it, where it sits among the runtime's other diagram models, and what it is good for. No code is needed to read this part.

Chapter 1. Draw.io diagrams as data

A Draw.io diagram is a file that people open in an editor. It is also, more often than its authors notice, a statement about a system: these are the parts, this one calls that one, this group belongs to that team. A file says it to the people who open it. A model says it to programs as well.

1.1. The style string in miniature

A shape's appearance, and a fair amount of its meaning, live in one attribute of the file:

```
rounded=1;whiteSpace=wrap;html=1;fillColor=#dae8fc;strokeColor=#6c8ebf;
```

Sometimes it is the only place the diagram says what a box *is*: `shape=mxgraph.aws4.lambda_function;`. A program that keeps the style as a string has learned nothing about it. `fillColor` is not queryable, not settable without string surgery, and not comparable between two diagrams.

That is the whole Draw.io modeling problem in miniature. The file holds structure, but in a form made for the editor, not for programs: a flat list of cells with parent ids, a style string per cell, properties as XML attributes on an `<object>` wrapper that appears only when a shape gets its first property, and links whose meaning depends on a prefix.

In this model the style is a map, in the order Draw.io wrote it, so `fillColor` is a key with a value. Unknown keys are kept, because a model that silently drops `sketch=1` is a model nobody runs twice. Properties are attributes, a map as well. Links are objects: a link to another page or another shape is a reference to that page or shape, so "what links here" is a query, not a text search.

1.2. The API is the loader, the model is the resource

The runtime already has a Java API for Draw.io documents, the subject of the *Draw.io* Zephyr Book. It knows the format's quirks: the page, model and root that are logically one thing, compression, documents embedded in PNG images, the `<object>` wrapper. It reads, navigates, changes and writes documents.

What a plain Java object tree cannot do is take part in a resource set: be referenced from another model, be saved as XMI, JSON or YAML, be compared with a generic tool, or share a type

hierarchy with other diagram notations. So the division is deliberate. The API loads and saves; the model is what it loads into. The runtime's capability framework wires the two together, so that a `.drawio` URI in a resource set resolves to a model without anybody writing glue.

1.3. What stays Draw.io

Most of what a Draw.io diagram has, every diagram has: shapes with bounds, connections between them, nesting, layers, labels, documentation. Those come from the runtime's diagram model, which this one extends. What stays Draw.io is small, and worth naming:

- the style vocabulary;
- the custom properties on the `<object>` wrapper;
- tags, which the in-browser viewer filters by;
- the link vocabulary: `data:page/id,` and `data:element/id,`, which turn a folder of files into a network of diagrams;
- custom actions and page animations, the JSON that shares the `link` attribute;
- page settings: size, grid, and whether the document is compressed.

1.4. Fidelity to the format, not to the pixels

A `.drawio` file loaded and saved through the model should open in the editor as its author left it, because Draw.io is the tool people will keep using, and a model that scrambles their files is worthless to them. That is a goal for each feature the model has; Chapter 7 lists what this edition does not keep yet.

The pixels are a different matter. The model has positions and sizes, not routes: Draw.io computes where an orthogonal edge bends, where it meets a shape and how a label wraps each time it draws. A picture of a diagram is drawn by Draw.io's own code, through the runtime's `drawio-playwright` module (Chapter 9).

Chapter 2. One family of diagram models

The model does not start from nothing. It extends two of the runtime's models, and that is where most of its value comes from.

2.1. A page is a slide is a diagram

- **The diagram model** has what every diagram notation has: elements with ids, names, documentation and attributes; nodes with bounds and ports; connections with waypoints; layers; nesting; and the semantic elements a shape depicts, from any other model.
- **The presentation model** extends it with decks and slides: a presentation is a diagram of slides, and a slide is a diagram.
- **The Draw.io model** extends both: a document is a presentation, a page is a slide, and layers, nodes and connections are their diagram namesakes.

So a program written against the diagram model reads a Draw.io page without knowing it is one. The runtime's ELK adapter lays out a Draw.io page because it lays out diagrams (Chapter 9). A converter that turns a diagram into SVG draws a Draw.io page too.

2.2. Why Draw.io is the default surface

The other notations are formats. Draw.io is a format and a tool that everybody can already open:

- **Free and open source**, on the web and on the desktop, as a Visual Studio Code extension and a Confluence plug-in, and self-hostable, which matters in organizations that will not approve a new modeling tool.
- **Thousands of shapes**, including the icon sets of the cloud providers, so a diagram that means something specific can look like it.
- **An in-browser viewer** that turns pages, shows and hides layers, filters by tags and runs custom actions, so a generated site gets interactive diagrams for free.
- **Properties, tooltips, libraries and templates**, editable by someone who has never heard of EMF.

The last point is the on-ramp. A diagram can be drawn by one person and connected to a domain model by another, later, without redrawing anything.

2.3. Give and take

Draw.io gives the family the ideas that were proven on it in Nasdanika over years: executable diagrams, semantic mapping by properties, documentation sites generated from diagrams, the cross-document link vocabulary, and geometric ordering of elements. None of them was really about mxGraph. On shared supertypes, each becomes a family feature.

The family gives Draw.io what it never had: layout with ELK, written once for all notations; generation from any model; and, PLANNED, three-way merge of diagrams that people and programs both edit, and routes in from other notations.

Chapter 3. What else there is

3.1. Draw.io itself

The source, not the rival, and the tool this model exists to keep people using. Draw.io has a JavaScript object model that lives in the editor and is gone when the tab closes. Its extension points (plug-ins, shape libraries, templates) extend the editor. Nothing in it is about what happens when a thousand diagrams need to be queried, checked, generated or connected to an architecture model, because that was never its job.

3.2. Parsers and one-off converters

Scripts that inflate the XML and walk it, the export modes of the Draw.io desktop application, converters to Mermaid or to images. Each solves one direction of one problem and produces a parse tree that lives for a few milliseconds. Few round-trip, none knows what a shape stands for, and many fail on the `<object>` wrapper, page links or a style key they have not seen before.

3.3. Diagrams as code

Mermaid, PlantUML, D2, Graphviz and the Structurizr DSL are text first, version-controlled, and good at what they do. Their limit is that the source is the artifact: there is no object graph to query, no stable identity to refer to from elsewhere, and references between diagrams are string matching. And they are not drawing tools for the people who own the content: a subject-matter expert will draw a box, and will not learn a DSL.

Structurizr is the closest relative in spirit: a model with views generated from it. The difference is direction. Structurizr is model first and shaped by C4, with diagrams as output; this model is diagram first, with mapping to models as an optional step, and its editor is the one people already have.

3.4. Modeling tools

Eclipse's GMF, Sirius, Graphiti and GLSP keep diagrams in models too, as the persistence format of their editors. Those formats are bound to their editors, and a shape without its semantic element is an orphan. Here the editor is Draw.io, and a shape that depicts nothing is still a

documented element.

3.5. The real competitor

A `.drawio` file committed to a repository that nothing reads, a PNG pasted into a wiki, and an architecture diagram that was accurate on the day it was drawn.

Chapter 4. What it is for

Some of these work today with the model and the runtime's other modules; most build on it and are **PLANNED**. Each says which.

4.1. Queries and checks

Style is a map and properties are attributes, so "every shape whose `fillColor` is `#d5e8d4`", "every connection that is not orthogonal" and "every system without an owner" are queries over a resource set, not regular expressions over XML. Tags and links are references, so "every element tagged legacy" and "everything that links to this component" are queries too. A check that every system on every diagram has an owner, run in a build, works today.

4.2. Generated diagrams

A program builds a Draw.io model, lays it out with ELK, saves it as a `.drawio` file, and renders it with Draw.io's own viewer as a PNG image that still carries the document (Chapter 9). The image goes into documentation; a person who wants to rearrange the diagram opens the image in Draw.io. This works today.

4.3. Restyling

Changing a palette across a set of diagrams, or applying a corporate template, is a model transformation instead of a person opening files: load, change style values, save. This works today.

4.4. Documentation sites

A `.drawio` file published as a navigable site, every shape a page, every connection a link, with the diagram embedded and interactive. Nasdanika core has done this for years; in the runtime it will be generated with its HTML and Bootstrap models, from the Draw.io model or from any diagram. **PLANNED**

4.5. Diagrams mapped to models

Shapes connected to the elements of other models through the diagram model's semantic elements: a box depicts a system in an architecture model, an arrow a dependency. The diagram model's book maps a diagram to the Ecore model it depicts. Mapping driven by properties, as Nasdanika core's semantic mapping does, is **PLANNED**.

4.6. Animated diagrams

Custom actions and page animations as typed objects whose steps point at cells, layers and tags by reference: highlight the elements that a trace touched, in the order it touched them; animate a payment through the systems it passes; show a sequence diagram's messages on the architecture diagram they run on. The vocabulary is in the model; loading and generating it is **PLANNED**. Recording such an animation as a video works today, with the `drawio-playwright` module.

4.7. Regeneration without losing human work

A generated diagram that a person arranged by hand should survive the next generation: model changes merged into the person's layout by id, new elements placed, removed ones removed. **PLANNED**

Part II: The How

The model in Java: adding it to a project, its classes, loading Draw.io documents into a resource set, reading and changing them, saving them, and generating a diagram that ELK lays out and Draw.io's viewer draws.

Chapter 5. Getting started

The model is an EMF model, defined in Xcore and generated into Java when the runtime is built. Using it takes a Maven dependency, a line in `module-info.java`, and a resource set.

5.1. Adding the model

The runtime's models have a bill of materials, `org.nasdanika.sdk.runtime.models:bom`, which manages their versions. Import it in `dependencyManagement`, and depend on `org.nasdanika.sdk.runtime.models.drawio:model` without a version.

The model is a named Java module:

Listing 1. The module declaration of the Draw.io model

```
module org.nasdanika.sdk.runtime.models.drawio {

    exports org.nasdanika.sdk.runtime.models.drawio;
    exports org.nasdanika.sdk.runtime.models.drawio.impl;
    exports org.nasdanika.sdk.runtime.models.drawio.resource;
    exports org.nasdanika.sdk.runtime.models.drawio.services;
    exports org.nasdanika.sdk.runtime.models.drawio.util;

    requires transitive org.nasdanika.sdk.runtime.models.presentation;
    requires transitive org.nasdanika.sdk.runtime.drawio;
    requires org.jsoup;

    provides EPackage with DrawioEPackageProvider;

    provides CapabilityFactory with DrawioCapabilityFactory;

    provides ResourceSetContributor with DrawioResourceSetContributor;

}
```

It requires the presentation model transitively, which brings the diagram model, the runtime's core model, EMF and the runtime's `common` module with it. It requires the runtime's Draw.io API transitively too, because the loader takes the API's documents. It provides three services: its `EPackage`, a capability factory that offers the contents handler, and a resource set contributor that registers the resource factory.

5.2. Files a resource set loads

`NasdanikaResourceSet.createAndConfigure()` creates a resource set and lets every module on the module path contribute to it. With the model on the module path, these load as a Draw.io `Document`:

File	What it is
`system.drawio`	A Draw.io document, compressed or not
`system.drawio.png`	A PNG image with the document embedded, as Draw.io exports it with *Include a copy of my diagram*
`system.drawio.svg`	An SVG image with the document in the `content` attribute of its root element, as Draw.io's editable SVG export writes it

The `.drawio.png` and `.drawio.svg` names are the convention of Draw.io's Visual Studio Code extension, which edits such images directly. A `.drawio` file can be saved; images cannot (Chapter 8).

5.3. How the right handler is found

The model registers the runtime's `ContentsHandlingResourceFactory` for the `drawio` extension, and for `png` and `svg` unless another factory has them already. The factory reads the dots in the file name as qualifiers, right to left, and asks the capability framework for a contents handler for each, starting with the leftmost: in `system.drawio.png`, the qualifiers are `png`, `drawio` and `system`, and the model's handler answers for `drawio`, reading the qualifier to its right to know the format. Another module can offer handlers for its own qualifiers in the same way, for example `system.c4.drawio` for a mapping of Draw.io documents to a C4 model.

5.4. What the build does

The Xcore definition is in `src/main/resources/org/nasdanika/sdk/runtime/models/drawio/drawio.xcore`. Maven generates the Java sources into `src-gen` with the Xtext Maven plugin. The generated API is in three packages: the interfaces in `org.nasdanika.sdk.runtime.models.drawio`, their implementations in `impl`, and the switch and adapter factory in `util`. The loader, the saver and the contents handler are hand-written, in `resource`; the services are in `services`.

Chapter 6. The model

6.1. The classes

Class	Extends	What it is
`Document`	presentation `Presentation`	The file: `host`, `agent`, `version`, `compressed`. Its children are its pages, and `pages` lists them typed.
`Page`	presentation `Slide`	A page: `dx`, `dy`, `pageWidth`, `pageHeight`, `gridSize`; the `tags` of the page. Its children are its layers, and `layers` lists them typed. The root cell's properties and tooltip are the page's attributes and documentation.
`ModelElement`, an interface	`LinkTarget`	What layers, nodes and connections have in common: `label`, `style`, `styleString`, `link`, `tags`.
`Layer`	diagram `Layer`, `ModelElement`	A layer, `visible` or not. Its children are nodes and connections.
`Node`	diagram `Node`, `ModelElement`	A shape: `bounds` relative to the parent node, `collapsed`, `visible`. Its children are the nodes and connections inside it.
`Connection`	diagram `Connection`, `ModelElement`	An edge: `source` and `target`, `waypoints`, `visible`. Its children are its labels, nodes that Draw.io places along it.
`Tag`	core `StringIdentity`	A tag of a page. Its id is its name; `elements` are the elements that have it.
`SimpleLink`, `CustomLink`, `SpelLink`	`AbstractLink`	The `link` attribute: a URL or text kept as it is, a link to a page or an element, or a SpEL expression.

Everything a diagram element has, these have too: `id`, `name`, `documentation`, `attributes`, `semanticElements`, `children`, `markers` and `uris`, from the diagram and core models.

6.2. What maps where

In the file	In the model
A cell's `id`	`id`
`value` or `label`	`label`, as written: HTML when the style has `html=1`. Its text is `name`
`tooltip`	`documentation`
`style`	`style`, an ordered map; `styleString` writes it back
Other attributes of the `<object>` wrapper: the properties	`attributes`
`tags`	`tags`, references to the page's `Tag` objects
`link`	`link`
`visible="0"`	`visible`
`mxGeometry` of a vertex	`bounds`
`Array` of points of an edge	`waypoints`
`source`, `target`	`source`, `target`

6.3. Label and name

A label in Draw.io is often HTML: `<b>Internet Banking System</b><br>Accounts and payments`. The model keeps it as written in `label`, so saving reproduces it, and puts its text in `name`, the name every diagram element has: `Internet Banking System Accounts and payments`. A program that reads diagrams generically reads `name`. A program that builds a Draw.io diagram can set only `name`, and saving writes it as the label.

6.4. Style

`style` is an EMF map, in the order of the file. A key without a value, such as `ellipse` at the start of `ellipse;whiteSpace=wrap;`, has a null value. `styleString` is derived from the map, and reads as Draw.io writes a style: each entry followed by a semicolon.

The map does not know what the keys mean. Typed styles, with features for the keys worth naming and enumerations for arrows, lines and jumps, as the runtime's Draw.io API has them, are **PLANNED**.

6.5. Links

Draw.io puts three kinds of things in the `link` attribute, so the model has one reference, `link`, to one of three kinds of link:

- **`CustomLink`**, for the Nasdanika link notation, which extends Draw.io's page links to elements and to other documents:

  ```
  data:page/id,containers
  data:page/name,System+Context
  data:element/id,id,context/banking
  data:element/name,overview.drawio#name,Systems/Mainframe
  ```

  The selectors are kept as written: `resource`, the document before `#`; `pageSelectorType` and `pageSelector`; `elementSelectorType` and `elementSelector`. A link to a page or an element of the same document is resolved, and `target` refers to it.

- **`SpelLink`**, for `data:spel,` links, whose expression is kept and not evaluated.
- **`SimpleLink`**, for everything else: URLs, other `data:` URIs, and custom actions, whose JSON is kept as text.

Chapter 7. Loading and reading

The listings work on `internet-banking.drawio`, a hand-written document after the C4 model's example. Its first page, *System Context*, has a person and two systems with properties and tags, connections between them, a link to the second page, and a *Notes* layer. Its second page, *Containers*, has a boundary that links back to the system on the first page, with two containers inside it.

7.1. Loading

Listing 2. Loading a document into a resource set (`DrawioTests.java`)

```
ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure(); ①
URI uri = URI.createURI(DrawioTests.class.getResource("internet-banking.drawio").toString());
Resource resource = resourceSet.getResource(uri, true); ②
Document document = (Document) resource.getContents().getFirst();

assertThat(document.getPages()).extracting(Page::getName).containsExactly("System Context",
"Containers");
Page context = document.getPages().getFirst();
assertThat(context.getLayers()).extracting(Layer::getName).containsExactly(null, "Notes"); ③
assertThat(context.getAttributes().map()).containsEntry("audience", "Architects"); ④
```

① A resource set to which every module on the module path has contributed: the model's resource factory and contents handler among them.

② The resource's contents is one `Document`.

③ The page's layers: the default layer, which has no name, and *Notes*.

④ The properties of the page's root cell are the page's attributes.

7.2. Reading elements

Listing 3. A node and a connection

```
Node banking = element(document, "banking");
assertThat(banking.getLabel()).isEqualTo("<b>Internet Banking System</b><br>Accounts and
payments"); ①
assertThat(banking.getName()).isEqualTo("Internet Banking System Accounts and payments");
assertThat(banking.getDocumentation()).isEqualTo("Lets customers view their accounts and make
payments"); ②
assertThat(banking.getAttributes().map()) ③
    .containsEntry("type", "software-system")
    .containsEntry("owner", "Digital Channels")
```

```
    .doesNotContainKeys("label", "tooltip", "tags", "link");
assertThat(banking.getStyle().get("fillColor")).isEqualTo("#1168bd"); ④
assertThat(banking.getStyleString()).startsWith("rounded=1;whiteSpace=wrap;html=1;");
assertThat(banking.getBounds().getWidth()).isEqualTo(160);

Connection calls = element(document, "calls");
assertThat(calls.getSource()).isSameAs(banking); ⑤
assertThat(calls.getTarget()).isSameAs(element(document, "mainframe"));
assertThat(calls.getWaypoints()).hasSize(1);
assertThat(banking.getOutgoingConnections()).extracting(DiagramElement::getName).containsExactly(
"Reads and writes");
```

① The label as written, HTML, and its text as the name.

② The tooltip is the documentation.

③ The properties are the attributes. Label, tooltip, tags and link have features of their own.

④ The style is a map; `styleString` is the style as Draw.io writes it.

⑤ Connections refer to their ends, and the ends know their connections.

`DrawioTests.element` finds an element by id anywhere in the document. Containment is the diagram model's: a page's children are its layers, a layer's children its nodes and connections, a node's children the nodes and connections inside it.

7.3. Tags and links

Listing 4. Tags and links as references

```
Page context = document.getPages().getFirst();
Tag internal = context.getTags().stream().filter(t -> "internal".equals(t.getId())).findFirst
().orElseThrow();
assertThat(internal.getElements()) ①
    .extracting(e -> ((DiagramElement) e).getId())
    .containsExactlyInAnyOrder("banking", "mainframe");

Node banking = element(document, "banking");
CustomLink toPage = (CustomLink) banking.getLink(); ②
assertThat(toPage.getTarget()).isSameAs(document.getPages().get(1));

CustomLink toElement = (CustomLink) ((Node) element(document, "boundary")).getLink(); ③
assertThat(toElement.getTarget()).isSameAs(banking);
assertThat(toElement.getPageSelector()).isEqualTo("context");

assertThat(banking.getReferrers(org.nasdanika.sdk.runtime.models.drawio.DrawioPackage.Literals.CU
STOM_LINK__TARGET)) ④
    .containsExactly(toElement);

SimpleLink web = (SimpleLink) ((Node) element(document, "api")).getLink(); ⑤
```

```
assertThat(web.getTarget()).isEqualTo("https://runtime.sdk.nasdanika.org");
```

① A tag knows its elements, and an element its tags.

② A page link, resolved to the page.

③ An element link to another page, resolved to the element; the selectors are kept as written.

④ "What links here", answered by the core model's `getReferrers`.

⑤ A URL, kept as it is.

7.4. What this edition does not load

The loader reads what the table in Chapter 6 lists. These are not kept yet, so a document that has them loses them when saved through the model:

- the end points of a connection without a source or a target;
- the offsets of connection labels, and label positions along a connection beyond the geometry of their nodes;
- connection points defined on a shape, other than as `exitX`, `exitY`, `entryX` and `entryY` in the style;
- the page's background, and settings of the `mxGraphModel` other than those in the table.

Custom actions are loaded as `SimpleLink` text, so they are kept.

Chapter 8. Changing and saving

A Draw.io model is changed like any EMF model, and saved like any resource.

Listing 5. Changing a style and a property, and saving (DrawioTests.java)

```
Node mainframe = element(document, "mainframe");
mainframe.getStyle().put("fillColor", "#d5e8d4"); ①
mainframe.getAttributes().put("status", "retiring");

File file = new File("target/saved/internet-banking.drawio");
Resource saved = resourceSet.createResource(URI.createFileURI(file.getAbsolutePath())); ②
saved.getContents().add(EcoreUtil.copy(document));
saved.save(null);

var drawio = org.nasdanika.sdk.runtime.drawio.Document.load(file); ③
var drawioMainframe = drawio.getModelElementById("mainframe");
assertThat(drawioMainframe.getStyle().get("fillColor")).isEqualTo("#d5e8d4");
assertThat(drawioMainframe.getProperty("status")).isEqualTo("retiring");
assertThat(drawio.getModelElementById("banking").getLink()).isEqualTo("data:page/id,containers");
assertThat(drawio.getModelElementById("boundary").getLink()).isEqualTo("data:element/id,id,contex
t/banking");
```

① A style value and an attribute, changed through their maps.

② A resource for a `.drawio` file, with a copy of the document.

③ The saved file, read with the runtime's Draw.io API: the change is there, and the links are written as they were read.

The test then loads the saved file into another resource set and compares it with what was saved: they are equal.

8.1. How a document is written

The saver creates a document with the runtime's Draw.io API and lets it write the XML:

- Pages, layers and nodes first, in containment order, with their ids, so that children refer to their parents.
- Then connections, which need both ends.
- Then links, which need their targets. A custom link is written with its selectors, as its author wrote it. One that a program created with a target and no selectors is written by id.
- An element without a label gets its name as the label. An element without an id gets one

from the API.

The document is compressed if its `compressed` feature says so.

8.2. Images

A `.drawio.png` or `.drawio.svg` resource loads, but does not save: an image is a picture of the document, and the model has no pictures. Save the document as a `.drawio` file, and render it with the runtime's `drawio-playwright` module, which draws it with Draw.io's own viewer and embeds the document in the image (Chapter 9).

Chapter 9. Generating, laying out and rendering

This chapter generates a diagram from a model, lays it out with ELK, saves it as a `.drawio` file, renders it with Draw.io's viewer, and loads the picture back. The model is the running JVM: the runtime's modules in the boot layer, and which of them requires which.

9.1. Generating

Listing 6. A page of modules (`GeneratedDiagramTests.java`)

```
/**
 * A page with a node per module of the runtime in the boot layer, and a connection per
 * {@code requires} between them.
 */
static Document modules() {
    DrawioFactory factory = DrawioFactory.eINSTANCE;
    Document document = factory.createDocument();
    Page page = factory.createPage();
    page.setId("modules");
    page.setName("Modules");
    page.getAttributes().put("elk.direction", "DOWN"); ①
    page.getAttributes().put("elk.edgeRouting", "ORTHOGONAL");
    document.getChildren().add(page);
    Layer layer = factory.createLayer();
    page.getChildren().add(layer);

    List<ModuleDescriptor> descriptors = ModuleLayer.boot().modules().stream()
        .filter(module -> module.getName().startsWith(PREFIX))
        .map(Module::getDescriptor)
        .sorted(Comparator.comparing(ModuleDescriptor::name))
        .toList();
    Map<String, Node> nodes = new LinkedHashMap<>();
    for (ModuleDescriptor descriptor : descriptors) {
        Node node = factory.createNode();
        node.setId(descriptor.name());
        node.setName(descriptor.name().substring(PREFIX.length())); ②
        boolean model = descriptor.name().startsWith(PREFIX + "models.");
        node.getStyle().put("rounded", "1"); ③
        node.getStyle().put("whiteSpace", "wrap");
        node.getStyle().put("fillColor", model ? "#dae8fc" : "#f5f5f5");
        node.getStyle().put("strokeColor", model ? "#6c8ebf" : "#666666");
        layer.getChildren().add(node);
        nodes.put(descriptor.name(), node);
    }
    for (ModuleDescriptor descriptor : descriptors) {
        for (ModuleDescriptor.Requires requires : descriptor.requires()) {
```

```
            Node target = nodes.get(requires.name());
            if (target != null) {
                Connection connection = factory.createConnection();
                connection.setSource(nodes.get(descriptor.name()));
                connection.setTarget(target);
                connection.getStyle().put("endArrow", "open");
                connection.getStyle().put("endFill", "0");
                if (requires.modifiers().contains(ModuleDescriptor.Requires.Modifier.TRANSITIVE))
{
                    connection.getStyle().put("dashed", "1"); ④
                }
                layer.getChildren().add(connection);
            }
        }
    }
    return document;
}
```

① ELK's layout options are attributes: top to bottom, with orthogonal routes.

② The name is the label, since no label is set.

③ The style as a map. Models are blue, the rest grey.

④ A `requires transitive` is dashed.

No node has bounds yet.

9.2. Laying out

The runtime's `diagram/elk` module lays out diagram model elements, and a Draw.io page is one. Layers are transparent to it: their nodes are laid out together.

Listing 7. Laying out, and saving

```
Document document = modules();
Page page = document.getPages().getFirst();
new ElkLayout().layout(page); ①
for (DiagramElement element : page.getLayers().getFirst().getChildren()) {
    if (element instanceof Connection connection) {
        anchor(connection);
    }
}

ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();
File file = directory.resolve("modules.drawio").toFile();
Resource resource = resourceSet.createResource(URI.createFileURI(file.getAbsolutePath()));
resource.getContents().add(document);
resource.save(null); ②
```

① ELK sizes the nodes to fit their names, places them, and routes the connections. The bounds and waypoints are written back into the model.

② A `.drawio` file. Draw.io opens it with the layout ELK computed.

ELK routes a connection from the outline of its source to the outline of its target, and writes the whole route. Draw.io wants the bends as waypoints, and the ends as points on the shapes, relative to their bounds:

Listing 8. From an ELK route to Draw.io's waypoints and connection points

```
/**
 * ELK routes a connection from the outline of its source to the outline of its target. Draw.io
 * takes the inner points as waypoints, and the ends as exit and entry points, relative to the
 * bounds of the nodes.
 */
static void anchor(Connection connection) {
    EList<Point> route = connection.getWaypoints();
    if (route.size() < 2) {
        return;
    }
    Point exit = route.removeFirst();
    Point entry = route.removeLast();
    Bounds source = ((Node) connection.getSource()).getBounds();
    Bounds target = ((Node) connection.getTarget()).getBounds();
    connection.getStyle().put("exitX", relative(exit.getX() - source.getX(), source.getWidth()));
    connection.getStyle().put("exitY", relative(exit.getY() - source.getY(), source.
getHeight()));
    connection.getStyle().put("entryX", relative(entry.getX() - target.getX(), target.
getWidth()));
    connection.getStyle().put("entryY", relative(entry.getY() - target.getY(), target.
getHeight()));
}

private static String relative(double offset, double size) {
    return String.valueOf(Math.max(0, Math.min(1, Math.round(offset / size * 1000) / 1000.0)));
}
```

9.3. Rendering

Listing 9. Rendering with Draw.io's viewer, and loading the picture

```
var drawio = org.nasdanika.sdk.runtime.drawio.Document.load(file); ①
Path png = rendering().withScale(2).png(drawio, directory.resolve("modules.drawio.png")); ②

Resource image = NasdanikaResourceSet.createAndConfigure()
    .getResource(URI.createFileURI(png.toAbsolutePath().toString()), true); ③
Page loaded = ((Document) image.getContents().getFirst()).getPages().getFirst();
```

```
assertThat(loaded.getName()).isEqualTo("Modules");
```

① The saved file, read with the runtime's Draw.io API.

② A PNG image at twice the size, drawn by Draw.io's own viewer in a browser driven by Playwright, through the `drawio-playwright` module, with the document embedded. The *Draw.io* Zephyr Book describes rendering in full.

③ The image is a Draw.io resource too: `.drawio.png` loads as the document it carries.

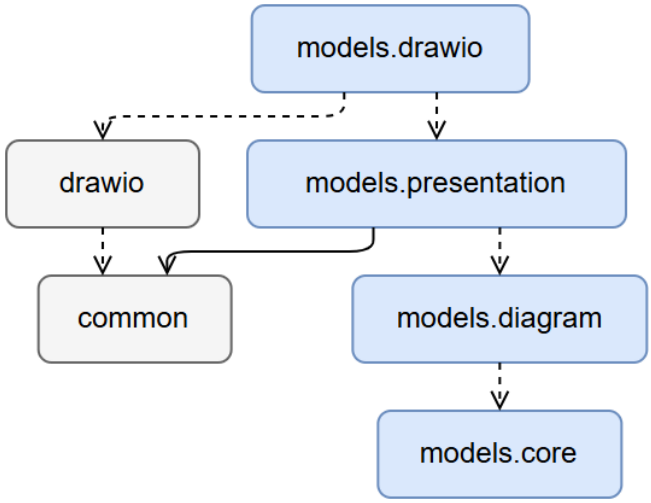


Figure 1. The modules of the test run (`modules.drawio.png`)

The figure shows the modules the test ran with: the Draw.io model and what it requires. The test's other dependencies, the ELK adapter and the rendering module, are on the class path in the test run, so they are not in the boot layer.

The same pipeline draws any model: a class hierarchy, a deployment, the systems of an architecture. A person who wants to arrange the result by hand opens the PNG in Draw.io; the next generation would overwrite their work, and merging the two is **PLANNED** (Chapter 10).

Part III: Looking Ahead

What the model is designed to do and does not do yet.

Chapter 10. Looking ahead

Everything in this chapter is **PLANNED**.

10.1. Custom actions and animations as objects

The model has the vocabulary of Draw.io's custom actions: show, hide, toggle, fade, wipe, pop, set and toggle style, highlight, flow, select, scroll, move the view, open a link, and change which tags are visible, with waits between steps. Their cells, layers and tags are references, not ids. `Actions` hold a sequence for a click, and `Animation` a sequence that a page runs when it is shown.

Loading the JSON into these objects and writing it back is next. Then an animation can be generated the way a diagram is: "highlight the elements that this trace touched, in order" or "show this sequence diagram's messages on the architecture diagram", checked before anything renders, because every step refers to elements that exist. The runtime's `drawio-playwright` module already records such animations as videos, so a generated animation becomes a generated video.

10.2. Typed styles

Features for the style keys worth naming, `fillColor`, `strokeColor`, `rounded`, `dashed`, `edgeStyle`, `startArrow` and `endArrow` among them, and enumerations for the closed vocabularies, derived from the style map so that the map stays the one copy of the truth.

10.3. Connection points as ports

Draw.io's `exitX`, `exitY`, `entryX` and `entryY` are ports by another name: a point on a shape where a connection attaches. As diagram model ports, they survive conversion to other notations and take part in layout.

10.4. Links across documents

Links into other documents loaded into the same resource set, resolved to their targets, so that a folder of diagrams is one graph and "what links here" works across files.

10.5. Merging generated and arranged diagrams

A diagram generated from a model and then arranged by a person, regenerated when the model changes, with the changes merged into the person’s layout by id: new elements placed, removed elements removed, the arrangement kept.

10.6. Mapping to models

Elements mapped to the objects of other models by their properties, as Nasdanika core’s semantic mapping does, through the diagram model’s semantic elements.

Appendix A: Reference

A.1. Model classes

Class	Extends	Features
`Document`	presentation `Presentation`	`host`, `agent`, `version`: strings. `compressed`: boolean. `pages`: `Page`, many, derived from `children`.
`Page`	presentation `Slide`, `LinkTarget`	`dx`, `dy`, `pageWidth`, `pageHeight`, `gridSize`: doubles. `tags`: `Tag`, many, contained. `animation`: `Animation`, contained. `layers`: `Layer`, many, derived from `children`.
`LinkTarget`, interface		What a custom link can point at: a page or a model element.
`ModelElement`, interface	`LinkTarget`	`label`: string. `style`: map of strings to strings. `styleString`: string, derived, read only. `link`: `AbstractLink`, contained. `tags`: `Tag`, many, opposite `elements`.
`Layer`	diagram `Layer`, `ModelElement`	
`Node`	diagram `Node`, `ModelElement`	`collapsed`: boolean. `visible`: boolean, true by default.
`Connection`	diagram `Connection`, `ModelElement`	`visible`: boolean, true by default.
`Tag`	core `StringIdentity`	`elements`: `ModelElement`, many, opposite `tags`.
`AbstractLink`, abstract		The `link` attribute.
`SimpleLink`	`AbstractLink`	`target`: string, kept as written.
`CustomLink`	`AbstractLink`	`resource`: string. `target`: `LinkTarget`. `pageSelectorType`, `elementSelectorType`: `SelectorType`. `pageSelector`, `elementSelector`: strings.
`SpellLink`	`AbstractLink`	`expression`: string.

Class	Extends	Features
`Actions`, `Animation`, `Action` and its subclasses		Custom actions and page animations. Not loaded or saved in this edition.

A.2. Enumerations

Enumeration	Literals
`SelectorType`	`ID`, `NAME`, `PROPERTY`
`TagsMatch`	`OR`, `AND`
`FlowMode`	`TOGGLE`, `START`, `STOP`

A.3. Java classes

Class	What it is for
`resource.DrawioLoader`	Loads a document of the runtime's Draw.io API into the model.
`resource.DrawioSaver`	Writes the model to a document of the API, which saves it as XML.
`resource.DrawioResourceContentsHandler`	The contents handler of Draw.io resources: XML, PNG or SVG.
`resource.Styles`	Style strings and style maps: `parse`, `format`.
`resource.Links`	Link text and link objects: `parse`, `format`.
`services.DrawioCapabilityFactory`	Offers the contents handler for the `drawio` qualifier.
`services.DrawioResourceSetContributor`	Registers the resource factory for `drawio`, `png` and `svg`.

Appendix B: Building this book, and its figure

This book is AsciiDoc in `models/drawio/model/src/docs/asciidoc`, built with the model after its tests have passed:

```
mvn -pl models/drawio/model -am verify                          # the model, its tests and this
book
mvn -pl models/drawio/model verify -Dzephyr.editor-notes=false   # without editor's notes
mvn -pl models/drawio/model verify -Dasciidoctor.skip=true       # without the book
```

The PDF and the single-page HTML are written to `target/book` and attached to the model's artifact with the classifier `book`; the test sources are attached with the classifier `test-sources`. The look comes from the runtime's `doc/zephyr` directory, shared by all the Nasdanika Zephyr Books.

Listings are tagged regions of the tests in `src/test/java/org/nasdanika/sdk/runtime/models/drawio/tests`. A listing line holds at most 96 characters, and a warning fails the build.

B.1. The tests that render

`GeneratedDiagramTests` renders with Draw.io's viewer in a browser. It takes the browser from the `playwright.channel` Maven property, `chrome` by default, or Playwright's Chromium if the property is empty, and it is skipped if no browser can be launched. The viewer is downloaded from jsDelivr, so the test needs the network.

B.2. The figure

The figure, `modules.drawio.png` in `src/test/resources/org/nasdanika/sdk/runtime/models/drawio/tests`, is the image that `GeneratedDiagramTests` writes to `target/generated`, copied. To change it, change the test, run it, and copy the image again. It is a PNG image with the Draw.io document embedded, so it also opens in Draw.io.

Appendix C: Glossary

Attribute

A string key and value on a model element, from the core model's `Attributed` mixin. A Draw.io property is an attribute.

Cell

Draw.io's word for an element of a page: the root, a layer, a shape or an edge. The model calls the last three model elements.

Contents handler

A capability that loads and saves the contents of a resource, chosen by the qualifiers of its file name.

Custom action

A step that Draw.io's viewer runs when a shape with a `data:action/json` link is clicked.

Custom link

A link to a page or an element in the Nasdanika notation, `data:page/⋯` or `data:element/⋯`.

Editable image

A PNG or SVG image that carries the Draw.io document it shows, so that Draw.io opens it for editing and a resource set loads it.

Layout

Positions and sizes of nodes and routes of connections, computed here by the Eclipse Layout Kernel.

Property

A named value of a Draw.io element, set in Draw.io's *Edit Data* dialog. In the model, an attribute.

Qualifier

A part of a file name between dots. In `system.drawio.png`, the qualifiers are `png`, `drawio` and `system`, read right to left.

Style

How Draw.io draws an element: a list of keys and values, such as `fillColor=#dae8fc`. In the model, an ordered map.

Tag

A name that elements of a page share, so that they can be shown, hidden and selected together.